第八章：

P271 8.1 使用 Camera2 实现定时拍照的功能。

1.运行界面：

2. 部分代码展示：

```
// 拍照
    private void takePhoto() {
        Log.d(TAG, "takePhoto()");
        if (null == mCameraDevice || !mPreviewView.isAvailable() || null == mPreviewSize) {
            return;
        }
        try {
            closePreview();
            mCaptureRequestBuilder = mCameraDevice.createCaptureRequest(CameraDevice.TEMPLATE_STILL_CAPTURE);
            // 设置预览的缓冲区大小
            SurfaceTexture texture = mPreviewView.getSurfaceTexture();
            texture.setDefaultBufferSize(mPreviewSize.getWidth(), mPreviewSize.getHeight());
            // 设置预览的 Surface
            List<Surface> surfaces = new ArrayList<>();
            Surface previewSurface = new Surface(texture);
            surfaces.add(previewSurface);
            mCaptureRequestBuilder.addTarget(previewSurface);
```

```
                // 设置拍照的 Surface
                Surface imageReaderSurface = mImageReader.getSurface();
                surfaces.add(imageReaderSurface);
                mCaptureRequestBuilder.addTarget(imageReaderSurface);
                // 创建捕捉会话
                mCameraDevice.createCaptureSession(surfaces, new
CameraCaptureSession.StateCallback() {
                    @Override
                    public void onConfigured(@NonNull CameraCaptureSession
cameraCaptureSession) {
                        Log.d(TAG, "CameraCaptureSession.onConfigured()");
                        mCaptureSession = cameraCaptureSession;
                        // 添加 ImageReader 监听器处理拍摄照片

mImageReader.setOnImageAvailableListener(mOnImageAvailableListener,
mBackgroundHandler);
                        mCaptureRequest = mCaptureRequestBuilder.build();
                        try {// 拍摄照片
                            mCaptureSession.capture(mCaptureRequest, null, null);
                        } catch (CameraAccessException e) {
                            e.printStackTrace();
                        }
                    }
                    @Override
                    public void onConfigureFailed(CameraCaptureSession
cameraCaptureSession) {
                        Toast.makeText(mContext, "摄像头配置失败",
Toast.LENGTH_LONG).show();
                    }
                }, mBackgroundHandler);
            } catch (CameraAccessException e) {
                e.printStackTrace();
            }
        }
        //定时拍照
        TextView tv = findViewById(R.id.show_time);
        CountDownTimer cdt = new CountDownTimer(5000, 1000)//参数 1：计时总时
间，参数 2：每次扣除时间数
        {
            int timer = 5;
            @Override
            public void onTick(long millisUntilFinished)
            {
                tv.setText(timer + "S");
```

```
                timer--;
                //定时时间到执行动作;
            }
            @Override
            public void onFinish() {
                tv.setText("");
                takePhoto();
            }
        }.start();
        // 处理拍摄的照片
        private void handlePhoto(ImageReader reader){
            Log.d(TAG, "handlePhoto()");
            byte[] photoByte = Util.imageToBytes(reader.acquireNextImage());
            mPhotoFile =
Util.creatFile(mContext.getExternalMediaDirs()[0].getAbsolutePath(),
mContext.getResources().getString(R.string.app_name), "jpg");
            // 保存拍摄的照片
            photoByte = Util.saveImage(photoByte, mPhotoFile, 90);
            // 将拍摄的照片显示在系统相册内
            Util.showInAlbum(mContext, mPhotoFile.getAbsolutePath());
            // 恢复预览
            startPreview();
            // 获取启动该 Activity 的 Intent
            Intent intent = getIntent();
            // 判断是否是通过外部 APP 隐式启动
            if
(intent.getAction().equals("android.media.action.IMAGE_CAPTURE")||intent.getAct
ion().equals("android.media.action.STILL_IMAGE_CAMERA")) {
                // 判断是否指定了照片保存路径的 Uri
                Uri uri = intent.getParcelableExtra(MediaStore.EXTRA_OUTPUT);
                if (uri != null) {
                    ContentResolver resolver = mContext.getContentResolver();
                    try {
                        // 向 Uri 指定路径的文件写入照片数据
                        ParcelFileDescriptor descriptor =
resolver.openFileDescriptor(uri, "rw");
                        FileOutputStream output = new
FileOutputStream(descriptor.getFileDescriptor());
                        output.write(photoByte);
                        descriptor.close();
                        output.close();
                    } catch (FileNotFoundException e) {
                        e.printStackTrace();
                    } catch (IOException e) {
```

```
                    e.printStackTrace();
                }
                setResult(RESULT_OK);
                finish();
            } else {
                // 设置照片压缩的参数
                BitmapFactory.Options options = new BitmapFactory.Options();
                options.inPreferredConfig = Bitmap.Config.RGB_565;
                options.inSampleSize = 16;
                // 压缩照片
                Bitmap bitmap = BitmapFactory.decodeFile(mPhotoFile.getAbsolutePath(), options);
                // 返回 RESULT_OK，并包含一个 Intent 对象，其中 Extra 中 key 为 data，value 是保存照片的 bitmap 对象。
                setResult(RESULT_OK, new Intent().putExtra("data", bitmap));
                finish();
            }
        } else {
            // 启动预览照片的 Activity
            intent = new Intent(MainActivity.this, PreviewActivity.class);
            intent.putExtra("path", mPhotoFile.getAbsolutePath());
            startActivity(intent); } }
```